```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

from google.colab import drive
drive.mount('/content/drive')
```

```
Mounted at /content/drive
```

```
car_accident =
pd.read_csv('/content/drive/MyDrive/Data_Science/US_Accidents_March23_
sampled_500k.csv')
car_accident.head(3)
```

```
{"type":"dataframe","variable_name":"car_accident"}
```

We will be working with 5,00,000 rows and 46 columns in this dataset.

```
car_accident.shape
```

```
(500000, 46)
```

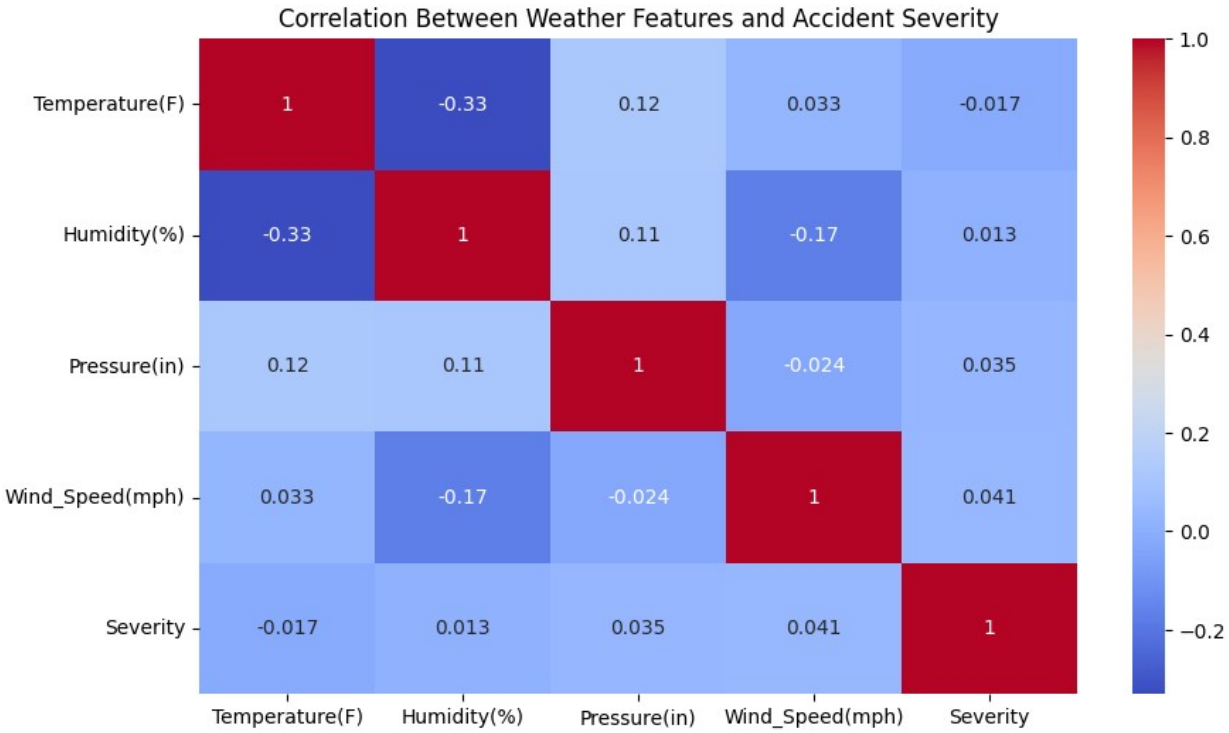
```
car_accident.columns
```

```
Index(['ID', 'Source', 'Severity', 'Start_Time', 'End_Time',
'Start_Lat',
       'Start_Lng', 'End_Lat', 'End_Lng', 'Distance(mi)',
'Description',
       'Street', 'City', 'County', 'State', 'Zipcode', 'Country',
'Timezone',
       'Airport_Code', 'Weather_Timestamp', 'Temperature(F)',
'Wind_Chill(F)',
       'Humidity(%)', 'Pressure(in)', 'Visibility(mi)',
'Wind_Direction',
       'Wind_Speed(mph)', 'Precipitation(in)', 'Weather_Condition',
'Amenity',
       'Bump', 'Crossing', 'Give_Way', 'Junction', 'No_Exit',
'Railway',
       'Roundabout', 'Station', 'Stop', 'Traffic_Calming',
'Traffic_Signal',
       'Turning_Loop', 'Sunrise_Sunset', 'Civil_Twilight',
'Nautical_Twilight',
       'Astronomical_Twilight'],
      dtype='object')
```

Correlation between severity and weather features

```
# Dropping rows with missing weather information & modifying DataFrame
directly
car_accident.dropna(subset=['Temperature(F)', 'Humidity(%)',
'Pressure(in)', 'Wind_Speed(mph)', 'Severity'], inplace=True)

# Correlation analysis between severity and weather features
correlation_matrix = car_accident[['Temperature(F)', 'Humidity(%)',
'Pressure(in)', 'Wind_Speed(mph)', 'Severity']].corr()
print("Correlation between severity and weather features:")
print(correlation_matrix)

# Heatmap visualization of correlations
plt.figure(figsize=(10, 6))
sns.heatmap(correlation_matrix, annot=True, cmap='coolwarm')
plt.title('Correlation Between Weather Features and Accident
Severity')
plt.show()
```

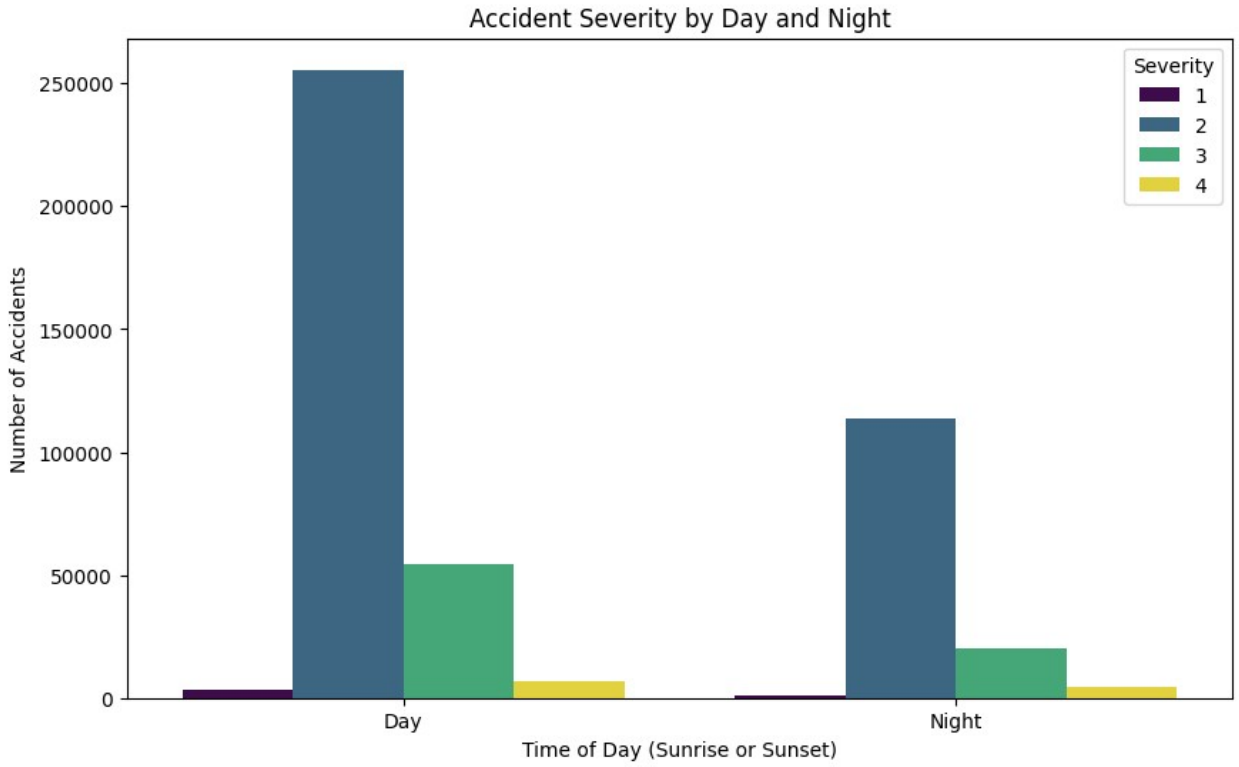
```
Correlation between severity and weather features:
                 Temperature(F)  Humidity(%)  Pressure(in)
Wind_Speed(mph)  \
Temperature(F)         1.000000    -0.330141      0.120246
0.032892
Humidity(%)           -0.330141     1.000000      0.113283        -
0.171253
Pressure(in)           0.120246     0.113283      1.000000        -
0.023810
Wind_Speed(mph)        0.032892    -0.171253     -0.023810
1.000000
Severity              -0.017024     0.013224      0.034818
0.041100

                 Severity
Temperature(F)  -0.017024
Humidity(%)      0.013224
Pressure(in)     0.034818
Wind_Speed(mph)  0.041100
Severity         1.000000
```



The weather features hold a very weak relation in accident severity. And have no dominant influence of any single feature. Only the wind speed has the highest correlation (0.045) with Severity. Still very weak relation.

Accident Severity difference between Day and Night

```
# Dropping rows with missing 'Sunrise_Sunset' and 'Severity'
car_accident.dropna(subset=['Sunrise_Sunset', 'Severity'],
inplace=True)

# Plot accident severity by day/night
plt.figure(figsize=(10, 6))
sns.countplot(data=car_accident, x='Sunrise_Sunset',
              hue='Severity', palette='viridis')
plt.xlabel('Time of Day (Sunrise or Sunset)')
plt.ylabel('Number of Accidents')
plt.title('Accident Severity by Day and Night')
plt.show()
```



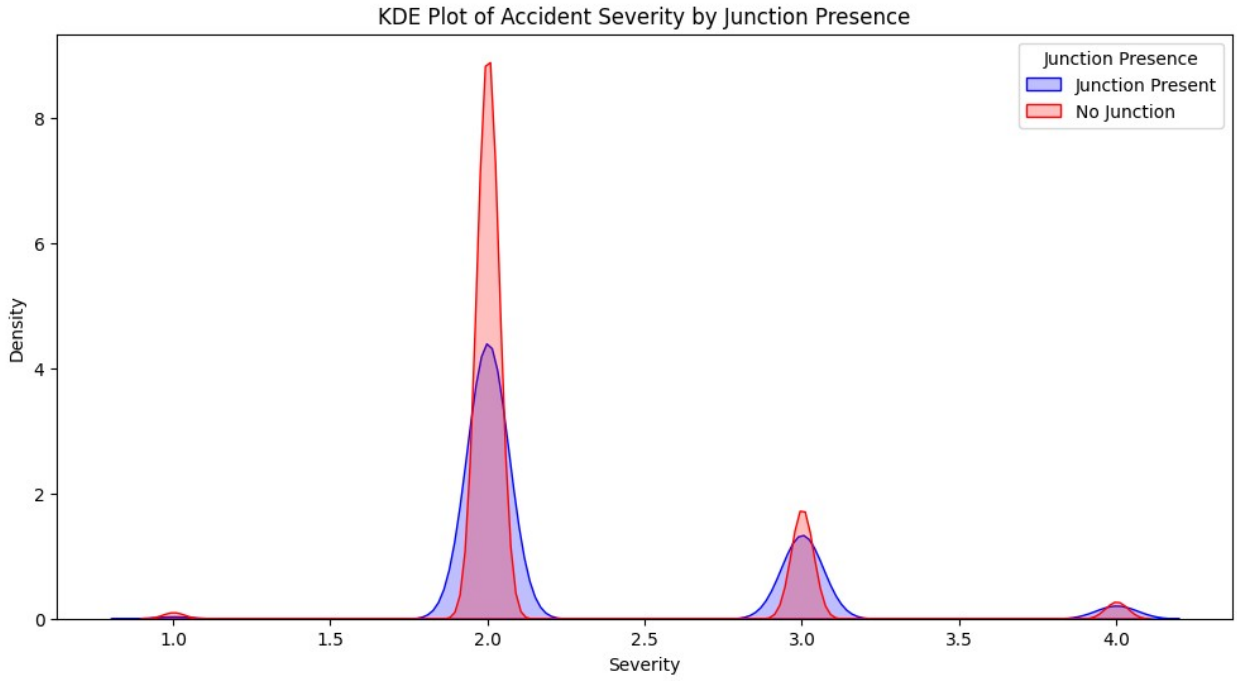
Most accidents happen during the day compared to night. Maybe it's because traffic volume is much higger during the day time. In both day and night time, Severity 2 is the dominating category. This visualization can help city planners increase safety features during daytime in high-risk areas

Density difference of Accident Severity depending on Junction Presence

```
# Dropping rows with missing key columns
car_accident.dropna(subset=['Junction', 'Severity'], inplace=True)

# Converting 'Junction' column into a string type.
#To treat the column as a categorical variable 'True' and 'False'.
car_accident['Junction'] = car_accident['Junction'].astype(str)

# Plotting KDE for accident severity based on junction presence
plt.figure(figsize=(12, 6))

sns.kdeplot(data=car_accident[car_accident['Junction'] == 'True'],
            x='Severity', label='Junction Present',
            shade=True, color='blue')
sns.kdeplot(data=car_accident[car_accident['Junction'] == 'False'],
            x='Severity', label='No Junction',
            shade=True, color='red')
```

```
plt.xlabel('Severity')
plt.ylabel('Density')
plt.title('KDE Plot of Accident Severity by Junction Presence')
plt.legend(title='Junction Presence')
plt.show()
```

```
<ipython-input-7-79d8b1e0c3d7>:11: FutureWarning:

`shade` is now deprecated in favor of `fill`; setting `fill=True`.
This will become an error in seaborn v0.14.0; please update your code.

  sns.kdeplot(data=car_accident[car_accident['Junction'] == 'True'],
<ipython-input-7-79d8b1e0c3d7>:14: FutureWarning:

`shade` is now deprecated in favor of `fill`; setting `fill=True`.
This will become an error in seaborn v0.14.0; please update your code.

  sns.kdeplot(data=car_accident[car_accident['Junction'] == 'False'],
```



The largest density peak for both Junction Present & No Junction occurs at Severity 2 means moderately severe but not life taking; might be because of urban areas with moderate traffic speeds

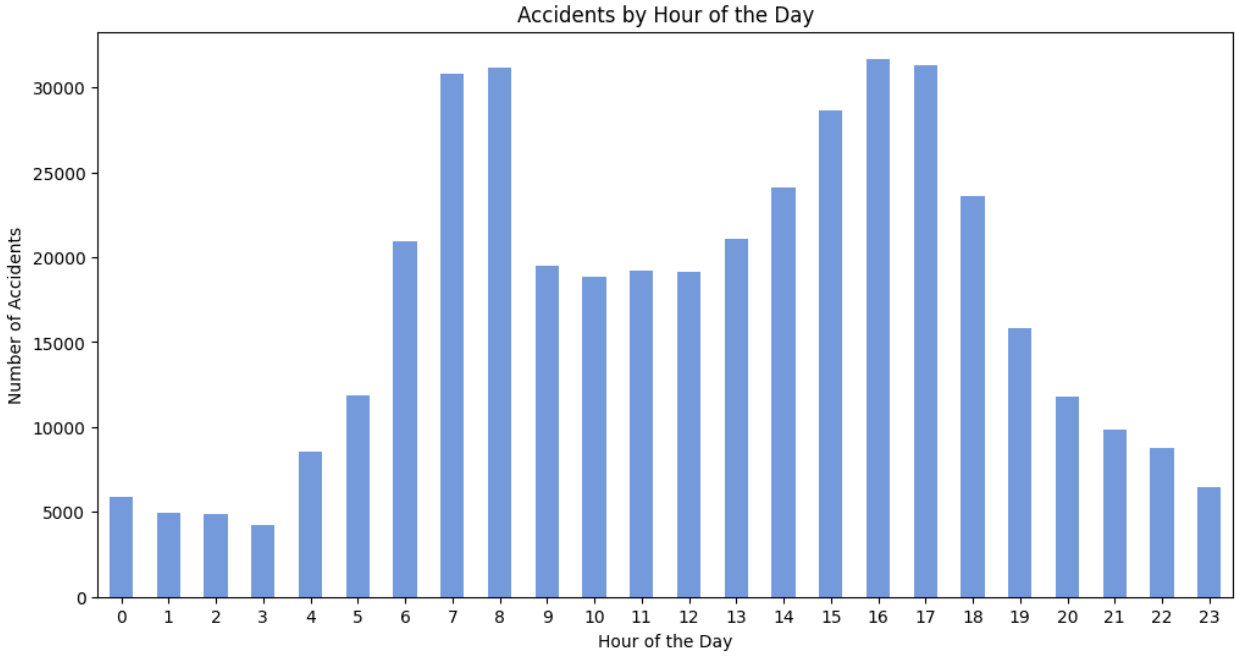
In both cases, there are higher densities with no junctions. The city planners can implements more roundabouts to slow down the traffic and reduce collisions

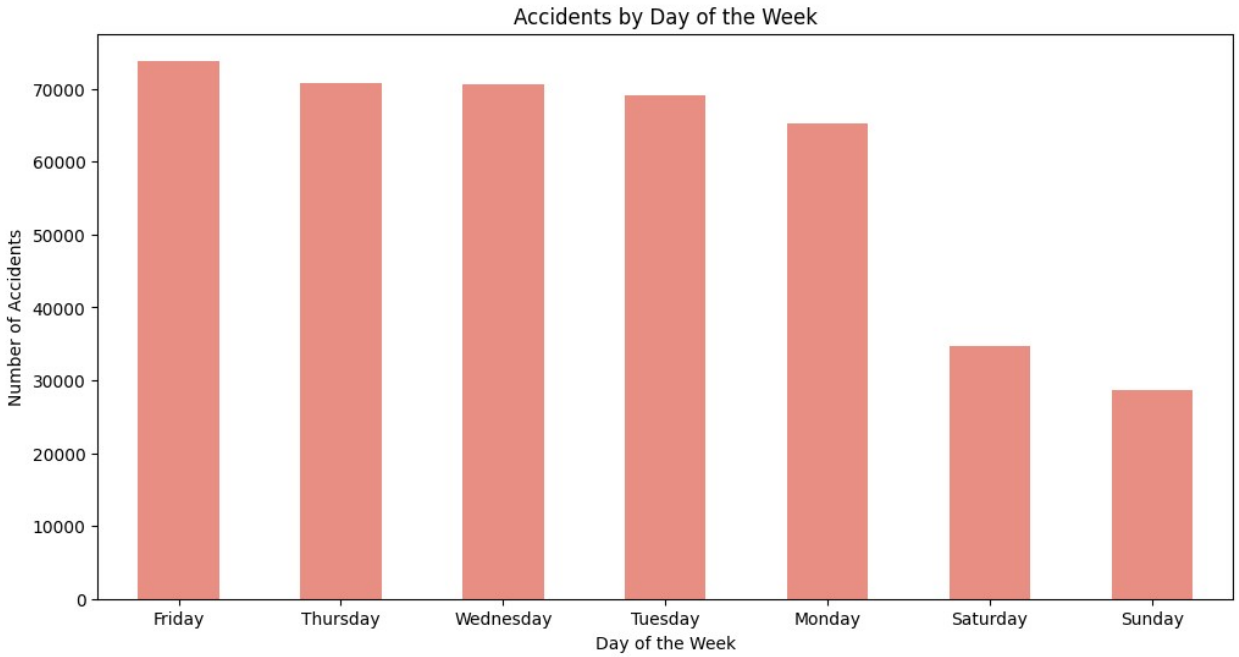
Number of accidents by hours of the day and days of the week to determine precautions.

```
# Converting 'Start_Time' to datetime and drop rows with missing
values
# errors='coerce' => replacing invalid datetime with NaT

car_accident['Start_Time'] =
pd.to_datetime(car_accident['Start_Time'], errors='coerce')
car_accident_time = car_accident.dropna(subset=['Start_Time'])

# Extracting temporal features & making Hour column and day of the
week into DayOfWeek column
car_accident_time['Hour'] = car_accident_time['Start_Time'].dt.hour
car_accident_time['DayOfWeek'] =
car_accident_time['Start_Time'].dt.day_name()


# Plotting accidents by hour of the day
plt.figure(figsize=(12, 6))
hourly_accidents =
car_accident_time['Hour'].value_counts().sort_index()
#counting number of accidents for each hour & sorting index to display
0-23
sns.barplot(x=hourly_accidents.index, y=hourly_accidents.values,
color='cornflowerblue', width=0.5)
plt.xlabel('Hour of the Day')
plt.ylabel('Number of Accidents')
plt.title('Accidents by Hour of the Day')
plt.show()


# Plotting accidents by day of the week
plt.figure(figsize=(12, 6))
weekday_accidents = car_accident_time['DayOfWeek'].value_counts()
#Counting number of accidents for each day of the week.
sns.barplot(x=weekday_accidents.index, y=weekday_accidents.values,
color='salmon',width=0.5)
plt.xlabel('Day of the Week')
plt.ylabel('Number of Accidents')
plt.title('Accidents by Day of the Week')
plt.show()
```

```
<ipython-input-8-b2ae93d00e36>:8: SettingWithCopyWarning:
A value is trying to be set on a copy of a slice from a DataFrame.
Try using .loc[row_indexer,col_indexer] = value instead

See the caveats in the documentation:
https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#
```

```
returning-a-view-versus-a-copy
  car_accident_time['Hour'] = car_accident_time['Start_Time'].dt.hour
<ipython-input-8-b2ae93d00e36>:9: SettingWithCopyWarning:
A value is trying to be set on a copy of a slice from a DataFrame.
Try using .loc[row_indexer,col_indexer] = value instead

See the caveats in the documentation:
https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#
returning-a-view-versus-a-copy
  car_accident_time['DayOfWeek'] =
car_accident_time['Start_Time'].dt.day_name()
```



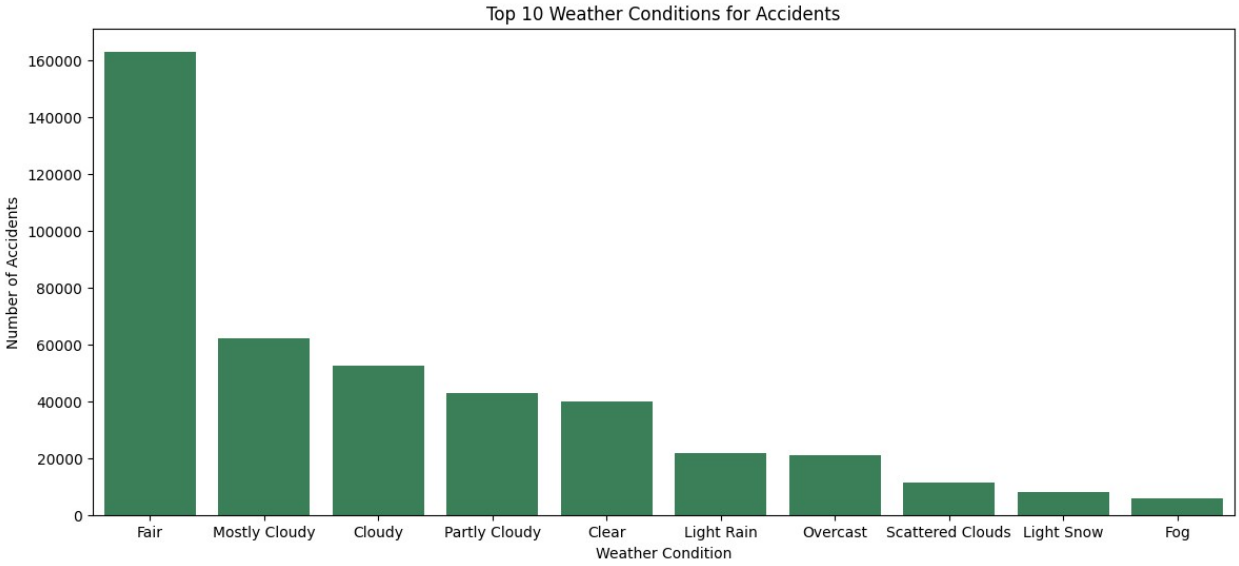


The peak hour of accidents is 7-8 AM & 4-5 PM indicating heavy traffic during commuting times. Authorities can implement more road safety measures at these hours

There is a significant drop in accidents rates on weekends & Fridays has the most traffic accidents reports. More traffic monitoring and Patrol presences are needed on working days

Top 10 Weather Conditions affecting more car accidents

```
# Filtering top 10 weather conditions after dropping missing rows
# nlargest(10) returning first 10 rows with the largest values in
columns with ascending false

top_weather_conditions =
car_accident['Weather_Condition'].dropna().value_counts().nlargest(10)

# Plotting accidents by Weather Condition using Seaborn
plt.figure(figsize=(14, 6))
sns.barplot(x=top_weather_conditions.index,
y=top_weather_conditions.values, color='seagreen')
plt.xlabel('Weather Condition')
plt.ylabel('Number of Accidents')
plt.title('Top 10 Weather Conditions for Accidents')
plt.show()
```



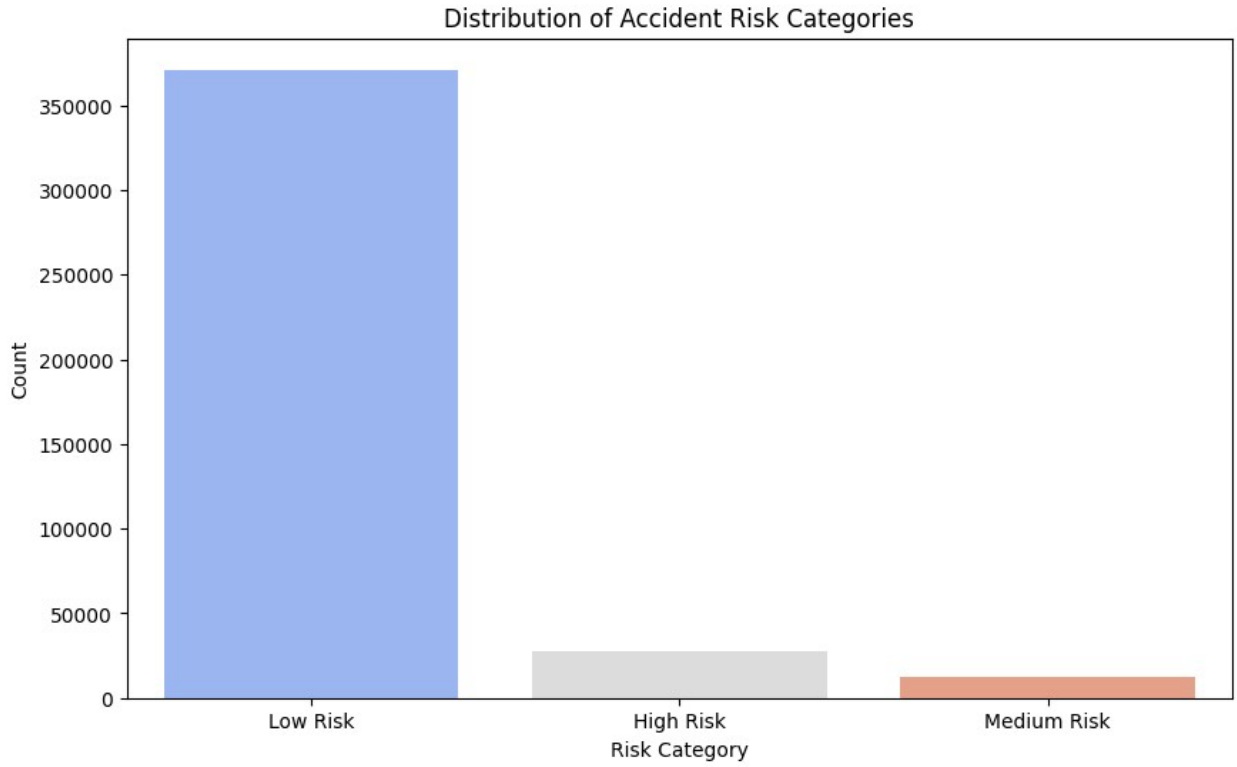
Most accidents happen during 'Fair' weather, which means weather is not a primary factor but the driving style, and traffic density are playing more role here

Visualizing accident risk category by weather, time of the day & traffic calming to aid road safety analysis

```
# Drop rows with missing key columns
car_accident.dropna(subset=['Start_Time', 'Weather_Condition',
'Traffic_Calming',
                            'Severity'], inplace=True)

# Parsing the Start_Time column into a datetime object
#Extracting the 'Hour' from Start_time column
car_accident['Start_Time'] =
pd.to_datetime(car_accident['Start_Time'], errors='coerce')
car_accident['Hour'] = car_accident['Start_Time'].dt.hour


# Defining the risk_category function based on the weather condition
and hours of the day
def risk_category(row):
    high_risk_weather = {'Fair', 'Mostly Cloudy'}
    medium_risk_weather = {'Cloudy', 'Partly Cloudy'}
    high_risk_hours = {7, 8}                    # 7 - 8 AM
    medium_risk_hours = {16, 17}                # 4 - 5 PM

    if row['Weather_Condition'] in high_risk_weather and row['Hour']
in high_risk_hours and not row['Traffic_Calming']:       #Traffic
```

```
calming measures are absent
        return 'High Risk'
    elif row['Weather_Condition'] in medium_risk_weather and
row['Hour'] in medium_risk_hours and not row['Traffic_Calming']:
        return 'Medium Risk'
    else:
        return 'Low Risk'


# Applying the risk_category function row-wise and creating a new
column named 'RiskCategory'
car_accident['RiskCategory'] = car_accident.apply(risk_category,
axis=1)

# Counting occurrences of each risk category
risk_counts = car_accident['RiskCategory'].value_counts()
print(risk_counts)

# Plotting the distribution of risk categories
plt.figure(figsize=(10, 6))
sns.barplot(x=risk_counts.index, y=risk_counts.values,
palette='coolwarm')
plt.xlabel('Risk Category')
plt.ylabel('Count')
plt.title('Distribution of Accident Risk Categories')
plt.show()
```

```
RiskCategory
Low Risk       371190
High Risk       27903
Medium Risk     12803
Name: count, dtype: int64

<ipython-input-12-2bf0da644e3c>:35: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be
removed in v0.14.0. Assign the `x` variable to `hue` and set
`legend=False` for the same effect.

  sns.barplot(x=risk_counts.index, y=risk_counts.values,
palette='coolwarm')
```



Most accidents happen under Low Risk conditions, suggesting that risk factors like severe weather and peak hours are mostly associated with higher severity accidents.

While High Risk accidents are fewer in number, they may still result in more severe outcomes

Geographical visualization of accidents across the USA

```
import folium
# Taking a random sample of 10,000 rows with valid location data
car_accident_sample = car_accident.dropna(subset=['Start_Lat',
'Start_Lng']).sample(n=10000, random_state=42)

USA_map = folium.Map(location=[39.8283, -98.5795],
                     zoom_start=5)

# Loop through each accident in the sampled dataset
for index, accident in car_accident_sample.iterrows():
#not using the index currently
    folium.CircleMarker(
        location=[accident['Start_Lat'],
                  accident['Start_Lng']],
# Latitude and Longitude
        radius=2, color='red',   fill=True,
```

```
        fill_color='red',
        fill_opacity=0.6).add_to(USA_map)

USA_map
```

```
<folium.folium.Map at 0x781e7100a500>
```

1.Urban Centers Are High-Risk Areas: Los Angeles, New York City, Miami, and Chicago have huge car accidents rates.

2. Heavy traffic volumns, complex road networks can be valid reasons.

3.The East Coast, shows a much denser pattern of accidents compared to the western regions

4.The red marks on the map often form linear patterns indicating major highways show frequent accidents

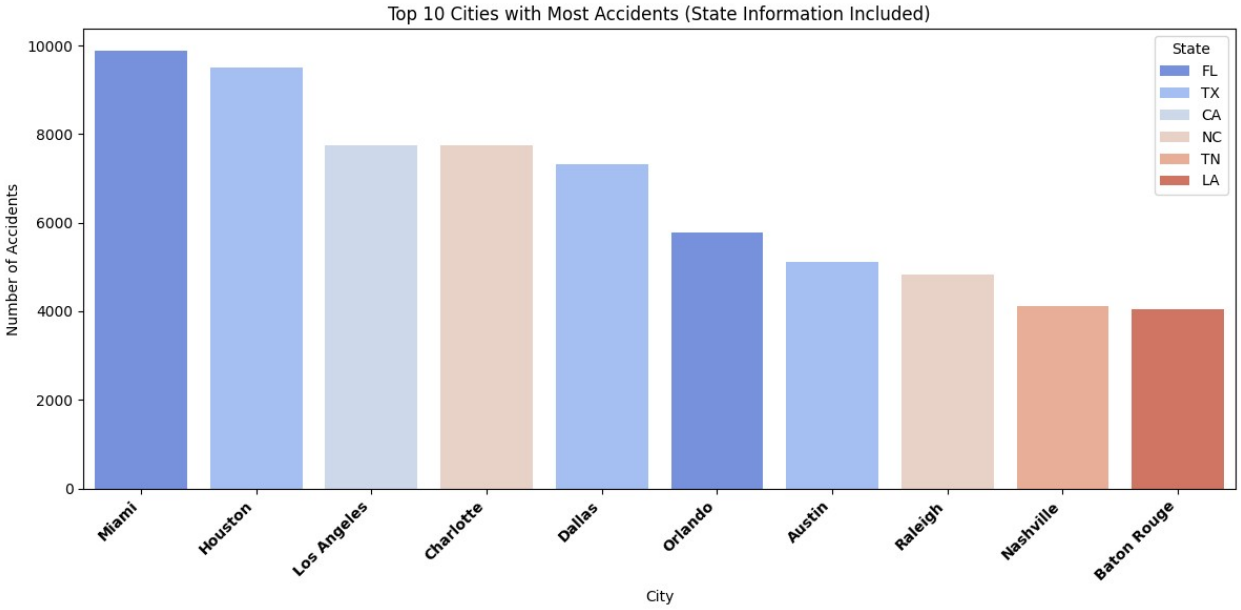
5.California, Florida, and Texas Are Critical Focus Areas in both rural & urban areas

Top 10 cities with most accidents

```
# Fill missing values in 'State' with 'Unknown' to flag them for
review
car_accident['State'] = car_accident['State'].fillna('Unknown')

# Group by 'City' and 'State' and count the number of accidents
city_state_grouped = car_accident.groupby(['City',
'State']).size().reset_index(name='Accident Count')

# Sort by Accident Count in descending order and select the top 10
cities
top_cities = city_state_grouped.sort_values('Accident Count',
ascending=False).head(10)

# Print the top 10 cities with their states and accident counts
print("Top 10 Cities with Most Accidents :")
print(top_cities)

# Visualization
plt.figure(figsize=(12, 6))
sns.barplot(data=top_cities, x='City', y='Accident Count',
hue='State',  palette='coolwarm')
plt.title("Top 10 Cities with Most Accidents (State Information
Included)")
plt.xlabel("City")
plt.ylabel("Number of Accidents")
plt.xticks(rotation=45, ha='right', fontsize=10, fontweight='bold',
color='black')
plt.legend(title="State", loc='upper right')
```

```
plt.tight_layout()
plt.show()
```

```
Top 10 Cities with Most Accidents :
            City State  Accident Count
6866        Miami    FL            9880
5033      Houston    TX            9512
6227  Los Angeles    CA            7746
1857    Charlotte    NC            7738
2580       Dallas    TX            7326
8140      Orlando    FL            5775
489        Austin    TX            5124
8918      Raleigh    NC            4831
7467    Nashville    TN            4126
637   Baton Rouge    LA            4045
```




```
missing_data = car_accident.isnull().sum()
missing_data[missing_data > 0].sort_values(ascending=True)
```

```
Description                 1
City                       19
Zipcode                   116
Timezone                  507
Street                    691
Airport_Code             1446
Astronomical_Twilight    1483
Civil_Twilight           1483
Sunrise_Sunset           1483
Nautical_Twilight        1483
Weather_Timestamp        7674
```

```
Pressure(in)               8928
Temperature(F)            10466
Weather_Condition         11101
Humidity(%)               11130
Wind_Direction            11197
Visibility(mi)            11291
Wind_Speed(mph)           36987
Wind_Chill(F)            129017
Precipitation(in)        142616
End_Lng                  220377
End_Lat                  220377
dtype: int64
```